

Task 1 - Polyp Characterization (CCE) with DenseNet121

Group member: Le Chef. Model: **DenseNet121, pretrained on ImageNet** (fine-tuned, binary head).

Chosen deliberately different from the group's ResNet18-from-scratch run (Accuracy 70.83%, Sensitivity 93.75%, Specificity 25.00%, F1 0.8108, AUC 0.7205) so the group's Table 1 gets a **Pretrained** row instead of a duplicate architecture.

DenseNet121 - model description (~half page)

DenseNet (Huang et al., 2017) connects each layer to **every other layer in a feed-forward fashion** within a 'dense block': layer ℓ receives as input the concatenation of the feature maps of all preceding layers, $x_{\ell} = H_{\ell}([x_0, x_1, \dots, x_{\ell-1}])$, rather than the sum used in ResNet's skip connections. This concatenation (not addition) is the key structural difference from ResNet.

Architecture. DenseNet121 has 4 dense blocks (6, 12, 24, 16 layers respectively), each block's layers using a bottleneck 1x1 conv + 3x3 conv (BN-ReLU-Conv order, 'pre-activation'), separated by transition layers (1x1 conv + 2x2 average pooling) that compress channel count (controlled by a 'growth rate' $k=32$, i.e. each layer adds k new feature maps to the collective state) and halve spatial resolution.

Why it fits this task. (1) Feature reuse: because every layer sees all earlier feature maps directly, the network needs far fewer parameters than a comparably deep ResNet/VGG to reach the same representational power - relevant given the CCE polyp dataset is small (few hundred images), where a large from-scratch net overfits immediately, as seen in the group's ResNet18-scratch run (train acc 100% by epoch 12, val stuck ~93-96%, test only 70.83%). (2) Implicit deep supervision: because early layers receive gradient directly from the loss through many short paths, DenseNet mitigates vanishing gradients without needing residual summation. (3) It is a standard baseline in medical image classification literature (e.g. CheXNet for chest X-rays) precisely because of its parameter efficiency on limited labeled data.

Trade-off to note in the report. Concatenation of feature maps is memory-hungry (activations must be kept for the whole block), so DenseNet121 trains slower / uses more VRAM per image than ResNet18 despite having fewer parameters (~8M vs ~11M) - worth mentioning if you compare training time in the report.

```
In [1]: !pip install torch torchvision torchaudio scikit-learn
```

Collecting torch

Downloading torch-2.13.0-cp313-cp313-manylinux_2_28_x86_64.whl.metadata (38 kB)

```
Collecting torchvision
  Downloading torchvision-0.28.0-cp313-cp313-manylinux_2_28_x86_64.whl.metadata
(5.6 kB)
Collecting torchaudio
  Downloading torchaudio-2.11.0-cp313-cp313-manylinux_2_28_x86_64.whl.metadata
(6.9 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28
_x86_64.whl.metadata (11 kB)
Collecting filelock (from torch)
  Downloading filelock-3.32.2-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python
3.13/site-packages (from torch) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in /opt/conda/lib/python3.13/si
te-packages (from torch) (82.0.1)
Collecting sympy>=1.13.3 (from torch)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch)
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages
(from torch) (3.1.6)
Collecting fsspec>=0.8.5 (from torch)
  Downloading fsspec-2026.7.0-py3-none-any.whl.metadata (10 kB)
Collecting cuda-toolkit==13.0.3 (from cuda-toolkit[cublas,cudart,cufft,cufile,cup
ti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Li
nux"->torch)
  Downloading cuda_toolkit-13.0.3.0-py2.py3-none-any.whl.metadata (17 kB)
Collecting cuda-bindings<14,>=13.0.3 (from torch)
  Downloading cuda_bindings-13.3.1-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_
28_x86_64.whl.metadata (2.5 kB)
Collecting nvidia-cudnn-cu13==9.20.0.48 (from torch)
  Downloading nvidia_cudnn_cu13-9.20.0.48-py3-none-manylinux_2_27_x86_64.whl.meta
data (1.9 kB)
Collecting nvidia-cusparselt-cu13==0.8.1 (from torch)
  Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-manylinux2014_x86_64.whl.meta
data (12 kB)
Collecting nvidia-nccl-cu13==2.29.7 (from torch)
  Downloading nvidia_nccl_cu13-2.29.7-py3-none-manylinux_2_18_x86_64.whl.metadata
(2.1 kB)
Collecting nvidia-nvshmem-cu13==3.4.5 (from torch)
  Downloading nvidia_nvshmem_cu13-3.4.5-py3-none-manylinux2014_x86_64.manylinux_2
_17_x86_64.whl.metadata (2.1 kB)
Collecting triton==3.7.1 (from torch)
  Downloading triton-3.7.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_6
4.whl.metadata (1.7 kB)
Collecting nvidia-cublas==13.1.1.3.* (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch)
  Downloading nvidia_cublas-13.1.1.3-py3-none-manylinux_2_27_x86_64.whl.metadata
(1.8 kB)
Collecting nvidia-cuda-nvrtc==13.0.88.* (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_syste
m == "Linux"->torch)
```

```

    Downloading nvidia_cuda_nvrtc-13.0.88-py3-none-manylinux2010_x86_64.manylinux_2
    _12_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cuda-runtime==13.0.96.* (from cuda-toolkit[cublas,cudart,cufft,
cufile,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_sys
tem == "Linux"->torch)
    Downloading nvidia_cuda_runtime-13.0.96-py3-none-manylinux2014_x86_64.manylinux
    _2_17_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cufft==12.0.0.61.* (from cuda-toolkit[cublas,cudart,cufft,cufil
e,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system =
= "Linux"->torch)
    Downloading nvidia_cufft-12.0.0.61-py3-none-manylinux2014_x86_64.manylinux_2_17
    _x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvjitlink<14,>=13.0.88 (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_syst
em == "Linux"->torch)
    Downloading nvidia_nvjitlink-13.3.33-py3-none-manylinux2010_x86_64.manylinux_2_
    12_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cufile==1.15.1.6.* (from cuda-toolkit[cublas,cudart,cufft,cufil
e,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system =
= "Linux"->torch)
    Downloading nvidia_cufile-1.15.1.6-py3-none-manylinux2014_x86_64.manylinux_2_17
    _x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cuda-cupti==13.0.85.* (from cuda-toolkit[cublas,cudart,cufft,cu
file,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_syste
m == "Linux"->torch)
    Downloading nvidia_cuda_cupti-13.0.85-py3-none-manylinux_2_25_x86_64.whl.metada
    ta (1.7 kB)
Collecting nvidia-curand==10.4.0.35.* (from cuda-toolkit[cublas,cudart,cufft,cufi
le,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system
== "Linux"->torch)
    Downloading nvidia_curand-10.4.0.35-py3-none-manylinux_2_27_x86_64.whl.metadata
    (1.7 kB)
Collecting nvidia-cusolver==12.0.4.66.* (from cuda-toolkit[cublas,cudart,cufft,cu
file,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_syste
m == "Linux"->torch)
    Downloading nvidia_cusolver-12.0.4.66-py3-none-manylinux_2_27_x86_64.whl.metada
    ta (1.8 kB)
Collecting nvidia-cusparsed==12.6.3.3.* (from cuda-toolkit[cublas,cudart,cufft,cuf
ile,cupti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system
== "Linux"->torch)
    Downloading nvidia_cusparsed-12.6.3.3-py3-none-manylinux2014_x86_64.manylinux_2_
    17_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvtx==13.0.85.* (from cuda-toolkit[cublas,cudart,cufft,cufile,c
upti,curand,cusolver,cusparsed,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system ==
"Linux"->torch)
    Downloading nvidia_nvtx-13.0.85-py3-none-manylinux1_x86_64.manylinux_2_5_x86_6
    4.whl.metadata (1.8 kB)

```

```
Collecting cuda-pathfinder>=1.4.2 (from cuda-bindings<14,>=13.0.3->torch)
  Downloading cuda_pathfinder-1.6.0-py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: numpy in /opt/conda/lib/python3.13/site-packages
(from torchvision) (2.5.1)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.1
3/site-packages (from torchvision) (12.3.0)
Requirement already satisfied: scipy>=1.10.0 in /opt/conda/lib/python3.13/site-pa
ckages (from scikit-learn) (1.18.0)
Collecting joblib>=1.4.0 (from scikit-learn)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: narwhals>=2.0.1 in /opt/conda/lib/python3.13/site-
packages (from scikit-learn) (2.24.0)
Collecting threadpoolctl>=3.5.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.13/site-
packages (from jinja2->torch) (3.0.3)
Downloading torch-2.13.0-cp313-cp313-manylinux_2_28_x86_64.whl (526.6 MB)
```

```

0.0/526.6 MB ? eta -:--:--
-- 30.7/526.6 MB 157.2 MB/s eta 0:00:04
-- 48.2/526.6 MB 128.3 MB/s eta 0:00:04
-- 51.4/526.6 MB 95.4 MB/s eta 0:00:05
-- 86.0/526.6 MB 107.6 MB/s eta 0:00:05
-- 93.3/526.6 MB 110.9 MB/s eta 0:00:04
-- 95.2/526.6 MB 79.0 MB/s eta 0:00:06
-- 122.2/526.6 MB 87.0 MB/s eta 0:00:05
-- 142.6/526.6 MB 88.8 MB/s eta 0:00:05
-- 174.1/526.6 MB 98.5 MB/s eta 0:00:04
-- 174.6/526.6 MB 87.5 MB/s eta 0:00:05
-- 186.6/526.6 MB 86.0 MB/s eta 0:00:04
-- 217.1/526.6 MB 90.0 MB/s eta 0:00:04
-- 234.9/526.6 MB 94.3 MB/s eta 0:00:04
-- 235.7/526.6 MB 84.3 MB/s eta 0:00:04
-- 264.0/526.6 MB 87.2 MB/s eta 0:00:04
-- 299.9/526.6 MB 87.9 MB/s eta 0:00:03
-- 330.3/526.6 MB 96.9 MB/s eta 0:00:03
-- 361.5/526.6 MB 111.2 MB/s eta 0:00:02
-- 362.0/526.6 MB 105.5 MB/s eta 0:00:02
-- 385.9/526.6 MB 100.4 MB/s eta 0:00:02
-- 423.6/526.6 MB 104.1 MB/s eta 0:00:01
-- 464.8/526.6 MB 125.7 MB/s eta 0:00:01
-- 501.2/526.6 MB 150.6 MB/s eta 0:00:01
-- 522.2/526.6 MB 140.5 MB/s eta 0:00:01
-- 526.4/526.6 MB 143.5 MB/s eta 0:00:01
-- 526.4/526.6 MB 143.5 MB/s eta 0:00:01
-- 526.6/526.6 MB 116.8 MB/s 0:00:05
```

```
Downloading cuda_toolkit-13.0.3.0-py2.py3-none-any.whl (2.5 kB)
Downloading nvidia_cudnn_cu13-9.20.0.48-py3-none-manylinux_2_27_x86_64.whl (366.2 MB)
```

```

0.0/366.2 MB ? eta -:--:--
-- 21.2/366.2 MB 105.1 MB/s eta 0:00:04
-- 64.2/366.2 MB 159.6 MB/s eta 0:00:02
-- 127.9/366.2 MB 212.1 MB/s eta 0:00:02
-- 206.8/366.2 MB 257.3 MB/s eta 0:00:01
-- 267.4/366.2 MB 273.3 MB/s eta 0:00:01
-- 308.8/366.2 MB 305.2 MB/s eta 0:00:01
-- 345.0/366.2 MB 294.4 MB/s eta 0:00:01
-- 366.0/366.2 MB 279.1 MB/s eta 0:00:01
```

```

366.2/366.2 MB 212.2 MB/s 0:00:01
Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-manylinux2014_x86_64.whl (170.1 MB)
0.0/170.1 MB ? eta -:--:--
12.1/170.1 MB 62.0 MB/s eta 0:00:03
36.4/170.1 MB 90.5 MB/s eta 0:00:02
78.9/170.1 MB 130.7 MB/s eta 0:00:01
114.3/170.1 MB 151.6 MB/s eta 0:00:01
135.5/170.1 MB 134.9 MB/s eta 0:00:01
170.1/170.1 MB 151.8 MB/s eta 0:00:01
170.1/170.1 MB 135.1 MB/s 0:00:01
Downloading nvidia_nccl_cu13-2.29.7-py3-none-manylinux_2_18_x86_64.whl (206.0 MB)
0.0/206.0 MB ? eta -:--:--
47.2/206.0 MB 239.6 MB/s eta 0:00:01
54.5/206.0 MB 135.4 MB/s eta 0:00:02
86.0/206.0 MB 142.6 MB/s eta 0:00:01
111.9/206.0 MB 140.5 MB/s eta 0:00:01
163.1/206.0 MB 162.1 MB/s eta 0:00:01
205.8/206.0 MB 177.3 MB/s eta 0:00:01
206.0/206.0 MB 155.9 MB/s 0:00:01
Downloading nvidia_nvshmem_cu13-3.4.5-py3-none-manylinux2014_x86_64.whl (60.4 MB)
0.0/60.4 MB ? eta -:--:--
39.3/60.4 MB 195.5 MB/s eta 0:00:01
60.4/60.4 MB 168.0 MB/s 0:00:00
Downloading triton-3.7.1-cp313-cp313-manylinux_2_27_x86_64.whl (197.7 MB)
0.0/197.7 MB ? eta -:--:--
31.7/197.7 MB 158.0 MB/s eta 0:00:02
78.6/197.7 MB 195.8 MB/s eta 0:00:01
91.2/197.7 MB 174.7 MB/s eta 0:00:01
115.3/197.7 MB 143.7 MB/s eta 0:00:01
149.4/197.7 MB 148.8 MB/s eta 0:00:01
188.7/197.7 MB 158.9 MB/s eta 0:00:01
197.7/197.7 MB 150.2 MB/s eta 0:00:01
197.7/197.7 MB 134.5 MB/s 0:00:01
Downloading cuda_bindings-13.3.1-cp313-cp313-manylinux_2_24_x86_64.whl (6.5 MB)
0.0/6.5 MB ? eta -:--:--
6.5/6.5 MB 152.3 MB/s 0:00:00
Downloading nvidia_cublas-13.1.1.3-py3-none-manylinux_2_27_x86_64.whl (423.1 MB)
0.0/423.1 MB ? eta -:--:--
16.8/423.1 MB 82.8 MB/s eta 0:00:05
49.5/423.1 MB 123.1 MB/s eta 0:00:04
87.0/423.1 MB 144.6 MB/s eta 0:00:03
139.5/423.1 MB 173.6 MB/s eta 0:00:02
163.8/423.1 MB 168.0 MB/s eta 0:00:02
210.2/423.1 MB 174.2 MB/s eta 0:00:02
239.6/423.1 MB 170.2 MB/s eta 0:00:02
295.2/423.1 MB 203.3 MB/s eta 0:00:01
345.0/423.1 MB 213.7 MB/s eta 0:00:01
400.8/423.1 MB 218.1 MB/s eta 0:00:01
423.1/423.1 MB 219.5 MB/s eta 0:00:01
423.1/423.1 MB 219.5 MB/s eta 0:00:01
423.1/423.1 MB 171.4 MB/s 0:00:02
Downloading nvidia_cuda_cupti-13.0.85-py3-none-manylinux_2_25_x86_64.whl (10.7 MB)
0.0/10.7 MB ? eta -:--:--

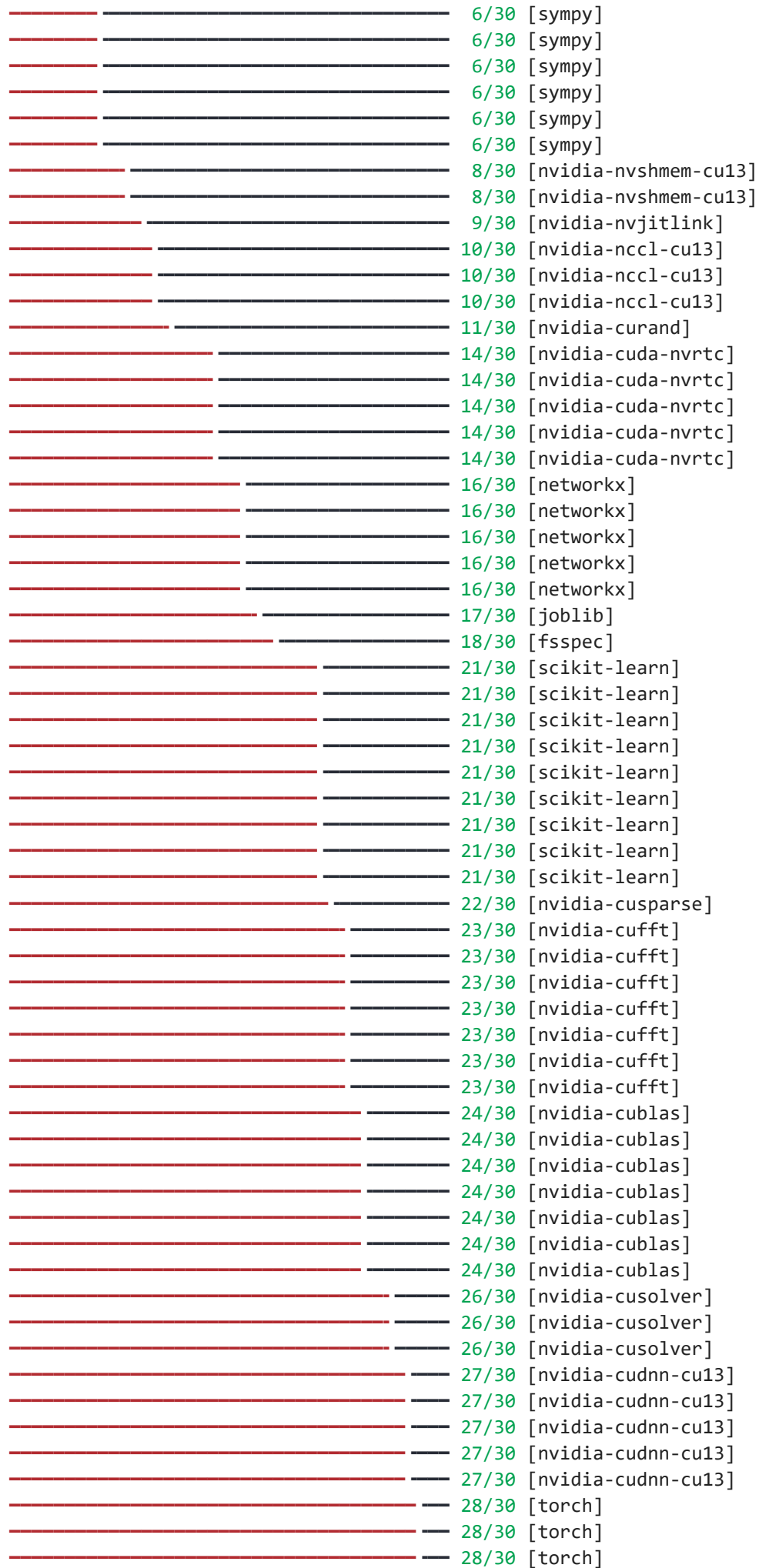
```

```

10.7/10.7 MB 175.9 MB/s 0:00:00
Downloading nvidia_cuda_nvrtc-13.0.88-py3-none-manylinux2010_x86_64.manylinux_2_1
2_x86_64.whl (90.2 MB)
0.0/90.2 MB ? eta -:--:--
35.9/90.2 MB 180.1 MB/s eta 0:00:01
60.8/90.2 MB 151.2 MB/s eta 0:00:01
83.1/90.2 MB 137.7 MB/s eta 0:00:01
90.2/90.2 MB 127.3 MB/s 0:00:00
Downloading nvidia_cuda_runtime-13.0.96-py3-none-manylinux2014_x86_64.manylinux_2
_17_x86_64.whl (2.2 MB)
0.0/2.2 MB ? eta -:--:--
2.2/2.2 MB 235.1 MB/s 0:00:00
Downloading nvidia_cufft-12.0.0.61-py3-none-manylinux2014_x86_64.manylinux_2_17_x
86_64.whl (214.1 MB)
0.0/214.1 MB ? eta -:--:--
0.8/214.1 MB 6.0 MB/s eta 0:00:36
53.5/214.1 MB 177.1 MB/s eta 0:00:01
107.0/214.1 MB 213.4 MB/s eta 0:00:01
140.5/214.1 MB 200.1 MB/s eta 0:00:01
200.3/214.1 MB 221.7 MB/s eta 0:00:01
213.9/214.1 MB 227.9 MB/s eta 0:00:01
214.1/214.1 MB 193.3 MB/s 0:00:01
Downloading nvidia_cufile-1.15.1.6-py3-none-manylinux2014_x86_64.manylinux_2_17_x
86_64.whl (1.2 MB)
0.0/1.2 MB ? eta -:--:--
1.2/1.2 MB 273.4 MB/s 0:00:00
Downloading nvidia_curand-10.4.0.35-py3-none-manylinux_2_27_x86_64.whl (59.5 MB)
0.0/59.5 MB ? eta -:--:--
29.4/59.5 MB 146.9 MB/s eta 0:00:01
59.5/59.5 MB 169.7 MB/s 0:00:00
Downloading nvidia_cusolver-12.0.4.66-py3-none-manylinux_2_27_x86_64.whl (200.9 M
B)
0.0/200.9 MB ? eta -:--:--
35.4/200.9 MB 182.1 MB/s eta 0:00:01
64.2/200.9 MB 165.0 MB/s eta 0:00:01
114.3/200.9 MB 189.9 MB/s eta 0:00:01
132.1/200.9 MB 179.2 MB/s eta 0:00:01
136.3/200.9 MB 135.8 MB/s eta 0:00:01
172.0/200.9 MB 154.0 MB/s eta 0:00:01
172.8/200.9 MB 124.0 MB/s eta 0:00:01
200.8/200.9 MB 132.5 MB/s eta 0:00:01
200.9/200.9 MB 119.9 MB/s 0:00:01
Downloading nvidia_cuspars-12.6.3.3-py3-none-manylinux2014_x86_64.manylinux_2_17
_x86_64.whl (145.9 MB)
0.0/145.9 MB ? eta -:--:--
50.3/145.9 MB 323.9 MB/s eta 0:00:01
77.9/145.9 MB 193.8 MB/s eta 0:00:01
130.3/145.9 MB 216.1 MB/s eta 0:00:01
145.9/145.9 MB 186.9 MB/s 0:00:00
Downloading nvidia_nvjitlink-13.3.33-py3-none-manylinux2010_x86_64.manylinux_2_12
_x86_64.whl (40.7 MB)
0.0/40.7 MB ? eta -:--:--
40.7/40.7 MB 219.8 MB/s 0:00:00
Downloading nvidia_nvtx-13.0.85-py3-none-manylinux1_x86_64.manylinux_2_5_x86_64.w
hl (148 kB)
Downloading torchvision-0.28.0-cp313-cp313-manylinux_2_28_x86_64.whl (7.7 MB)
0.0/7.7 MB ? eta -:--:--

```

[illegible]



Successfully installed cuda-bindings-13.3.1 cuda-pathfinder-1.6.0 cuda-toolkit-13.0.3.0 filelock-3.32.2 fsspec-2026.7.0 joblib-1.5.3 mpmath-1.3.0 networkx-3.6.1 nvidia-cublas-13.1.1.3 nvidia-cuda-cupti-13.0.85 nvidia-cuda-nvrtc-13.0.88 nvidia-cuda-runtime-13.0.96 nvidia-cudnn-cu13-9.20.0.48 nvidia-cufft-12.0.0.61 nvidia-cufile-1.15.1.6 nvidia-curand-10.4.0.35 nvidia-cusolver-12.0.4.66 nvidia-cusparse-12.6.3.3 nvidia-cusparselt-cu13-0.8.1 nvidia-nccl-cu13-2.29.7 nvidia-nvjitlink-13.3.33 nvidia-nvshmem-cu13-3.4.5 nvidia-nvtx-13.0.85 scikit-learn-1.9.0 sympy-1.14.0 threadpoolctl-3.6.0 torch-2.13.0 torchaudio-2.11.0 torchvision-0.28.0 triton-3.7.1

```
In [2]: import os, random, copy
import numpy as np
import torch
import torch.nn as nn
import torch.utils.data as data
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from torchvision.models import densenet121
from PIL import Image
import matplotlib.pyplot as plt
from sklearn.metrics import (confusion_matrix, roc_curve, auc, precision_recall_
                             f1_score, accuracy_score)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)

seed = 27
torch.manual_seed(seed); random.seed(seed); np.random.seed(seed)

# Hyper-parameters
batch_size = 32
n_epochs = 40          # upper bound, early stopping will cut this short
learning_rate = 1e-4
patience = 5          # exercise requires Early Stopping, patience 4/5/6 -> u
```

cuda

Data

Same folder / split-file layout as the teacher's `polypcharacterization_cnns.ipynb` template (`./PCData/Neoplasia` , `./PCData/Non-neoplasia` , `train_data.txt` / `valid_data.txt` / `test_data.txt`). Reused as-is so results are directly comparable to your teammate's run.

```
In [4]: neoplastic_folder = './PCData/Neoplasia'
nonneoplastic_folder = './PCData/Non-neoplasia'

nonneoplastic_data = [(os.path.join(nonneoplastic_folder, f), 0) for f in os.lis
neoplastic_data = [(os.path.join(neoplastic_folder, f), 1) for f in os.listdir(n
all_data = neoplastic_data + nonneoplastic_data

def load_split(txt_path):
    ids = set()
    with open(txt_path, 'r') as f:
        for line in f:
            ids.add(line.strip().split(',')[0])
    split = []
    for filepath, label in all_data:
        sdkid = os.path.basename(filepath).split('_')[0]
        if sdkid in ids:
            split.append((filepath, label))
    return split

train_data = load_split('train_data.txt')
valid_data = load_split('valid_data.txt')
test_data = load_split('test_data.txt')
print(f'train={len(train_data)} valid={len(valid_data)} test={len(test_data)}')
```

train=1667 valid=562 test=569

```
In [5]: class CustomDataset(Dataset):
    def __init__(self, data, transform=None):
        self.data = data
        self.transform = transform
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        img_path, label = self.data[idx]
        image = Image.open(img_path).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, label, os.path.basename(img_path)

# Light augmentation on train only (helps a pretrained net generalise on a small
# without it DenseNet121 will just as easily overfit as the scratch ResNet18 did
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])
eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])

train_dataset = CustomDataset(train_data, transform=train_transform)
valid_dataset = CustomDataset(valid_data, transform=eval_transform)
test_dataset = CustomDataset(test_data, transform=eval_transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

Model: DenseNet121 (pretrained), binary head

```
In [6]: pc_model = densenet121(weights='IMAGENET1K_V1')
pc_model.classifier = nn.Linear(pc_model.classifier.in_features, 1) # binary Lo
pc_model = pc_model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(pc_model.parameters(), lr=learning_rate, betas=(0.9
```

Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to /home/ucload/.cache/torch/hub/checkpoints/densenet121-a639ec97.pth

```
0%|          | 0.00/30.8M [00:00<?, ?B/s]
64%|██████    | 19.9M/30.8M [00:00<00:00, 208MB/s]
100%|██████████| 30.8M/30.8M [00:00<00:00, 238MB/s]
```

Train / validate with Early Stopping (patience = 5)

```
In [7]: def run_epoch(model, dataloader, criterion, optimizer=None):
    is_train = optimizer is not None
    model.train() if is_train else model.eval()
    total_loss, correct, n = 0.0, 0, 0
    all_probs, all_targets = [], []
    ctx = torch.enable_grad() if is_train else torch.no_grad()
    with ctx:
        for images, targets, _ in dataloader:
            images = images.to(device)
            targets = targets.to(device).float().view(-1, 1)
            outputs = pc_model_forward = model(images)
            loss = criterion(outputs, targets)
            if is_train:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            probs = torch.sigmoid(outputs)
            preds = (probs > 0.5).float()
            correct += (preds == targets).sum().item()
            n += targets.size(0)
            total_loss += loss.item() * targets.size(0)
            all_probs.extend(probs.detach().cpu().numpy().ravel().tolist())
            all_targets.extend(targets.detach().cpu().numpy().ravel().tolist())
    return total_loss/n, 100*correct/n, np.array(all_probs), np.array(all_targets)

Train_Loss, Train_Acc, Val_Loss, Val_Acc = [], [], [], []
best_val_loss = float('inf')
best_state = None
epochs_no_improve = 0

for epoch in range(n_epochs):
    train_loss, train_acc, _, _ = run_epoch(pc_model, train_loader, criterion, optimizer)
    val_loss, val_acc, _, _ = run_epoch(pc_model, val_loader, criterion, optimizer)
    Train_Loss.append(train_loss); Train_Acc.append(train_acc)
    Val_Loss.append(val_loss); Val_Acc.append(val_acc)
    print(f'Epoch [{epoch+1}/{n_epochs}] Train Acc {train_acc:.2f}% Loss {train_loss:.4f} Val Acc {val_acc:.2f}% Loss {val_loss:.4f}')

    if val_loss < best_val_loss - 1e-4:
        best_val_loss = val_loss
        best_state = copy.deepcopy(pc_model.state_dict())
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print(f'Early stopping at epoch {epoch+1} (patience={patience})')
            break

pc_model.load_state_dict(best_state) # restore best checkpoint before evaluation
```

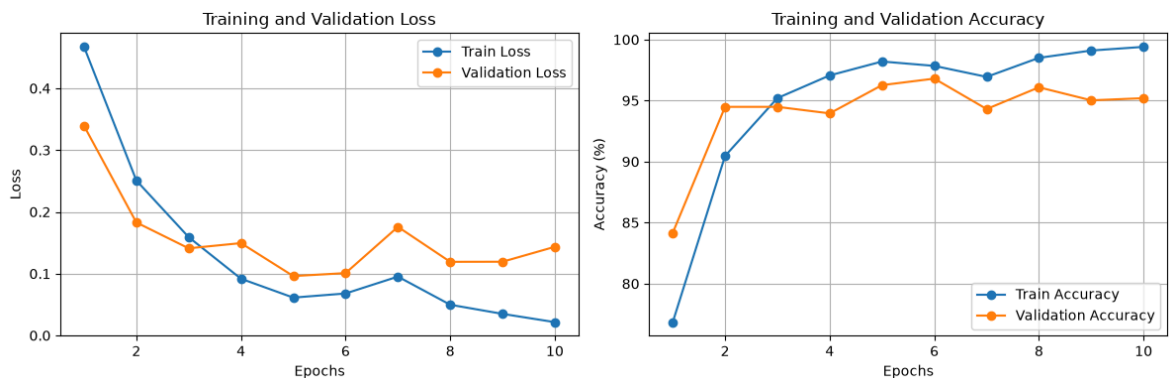
```
Epoch [1/40] Train Acc 76.84% Loss 0.4671 | Val Acc 84.16% Loss 0.3392
Epoch [2/40] Train Acc 90.46% Loss 0.2508 | Val Acc 94.48% Loss 0.1829
Epoch [3/40] Train Acc 95.20% Loss 0.1591 | Val Acc 94.48% Loss 0.1411
Epoch [4/40] Train Acc 97.06% Loss 0.0918 | Val Acc 93.95% Loss 0.1495
Epoch [5/40] Train Acc 98.20% Loss 0.0613 | Val Acc 96.26% Loss 0.0961
Epoch [6/40] Train Acc 97.84% Loss 0.0680 | Val Acc 96.80% Loss 0.1010
Epoch [7/40] Train Acc 96.94% Loss 0.0952 | Val Acc 94.31% Loss 0.1758
Epoch [8/40] Train Acc 98.50% Loss 0.0497 | Val Acc 96.09% Loss 0.1192
Epoch [9/40] Train Acc 99.10% Loss 0.0349 | Val Acc 95.02% Loss 0.1193
Epoch [10/40] Train Acc 99.40% Loss 0.0215 | Val Acc 95.20% Loss 0.1435
Early stopping at epoch 10 (patience=5)
```

Out[7]: <All keys matched successfully>

Graph 1 - Train vs Validation Loss / Accuracy (same layout as teammate's plot)

```
In [8]: epochs_ran = range(1, len(Train_Acc)+1)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(epochs_ran, Train_Loss, marker='o', label='Train Loss')
axes[0].plot(epochs_ran, Val_Loss, marker='o', label='Validation Loss')
axes[0].set_xlabel('Epochs'); axes[0].set_ylabel('Loss'); axes[0].set_title('Tra
axes[0].legend(); axes[0].grid(True)

axes[1].plot(epochs_ran, Train_Acc, marker='o', label='Train Accuracy')
axes[1].plot(epochs_ran, Val_Acc, marker='o', label='Validation Accuracy')
axes[1].set_xlabel('Epochs'); axes[1].set_ylabel('Accuracy (%)'); axes[1].set_ti
axes[1].legend(); axes[1].grid(True)
plt.tight_layout(); plt.savefig('densenet121_training_curves.png', dpi=150); plt
```



Test-set evaluation: Accuracy, Sensitivity, Specificity, F1, AUC (Table 1 row)

```
In [9]: test_loss, test_acc, test_probs, test_targets = run_epoch(pc_model, test_loader,
test_preds = (test_probs > 0.5).astype(int)

cm = confusion_matrix(test_targets, test_preds) # rows=true [Non-neo, Neo], col
tn, fp, fn, tp = cm.ravel()

accuracy = 100 * (tp + tn) / (tp + tn + fp + fn)
sensitivity = 100 * tp / (tp + fn) # recall for Neoplasia (positive class)
specificity = 100 * tn / (tn + fp)
f1 = f1_score(test_targets, test_preds)
fpr, tpr, _ = roc_curve(test_targets, test_probs)
roc_auc = auc(fpr, tpr)

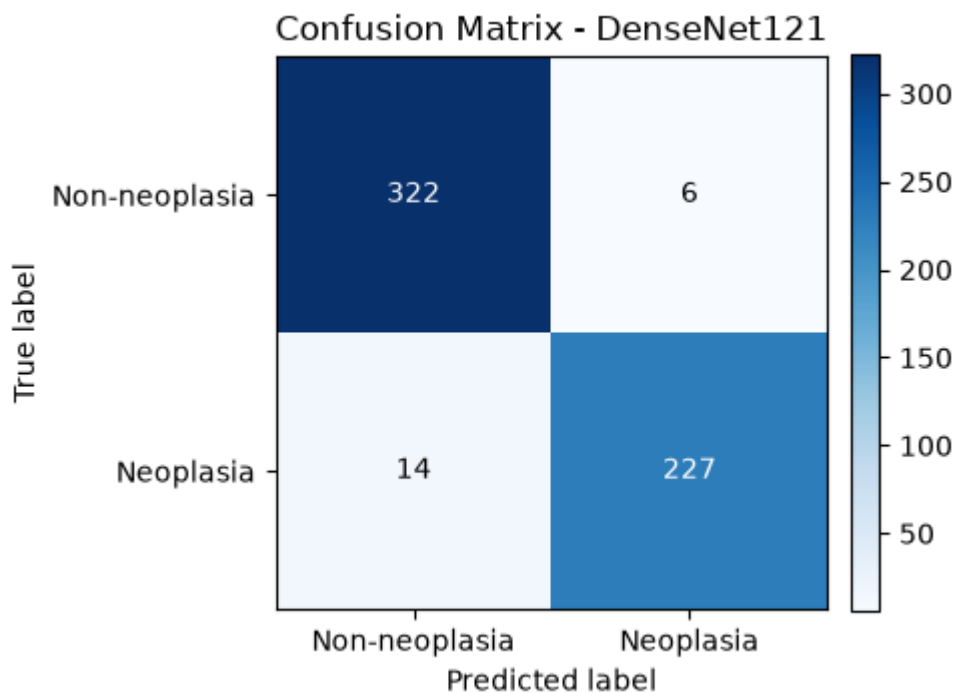
print('Model: DenseNet121 | Pretrained')
print('-'*40)
print(f'Accuracy (%) : {accuracy:.2f}')
print(f'Sensitivity (%) : {sensitivity:.2f}')
print(f'Specificity (%) : {specificity:.2f}')
print(f'F1 Score : {f1:.4f}')
print(f'AUC : {roc_auc:.4f}')
```

Model: DenseNet121 | Pretrained

 Accuracy (%) : 96.49
 Sensitivity (%) : 94.19
 Specificity (%) : 98.17
 F1 Score : 0.9578
 AUC : 0.9898

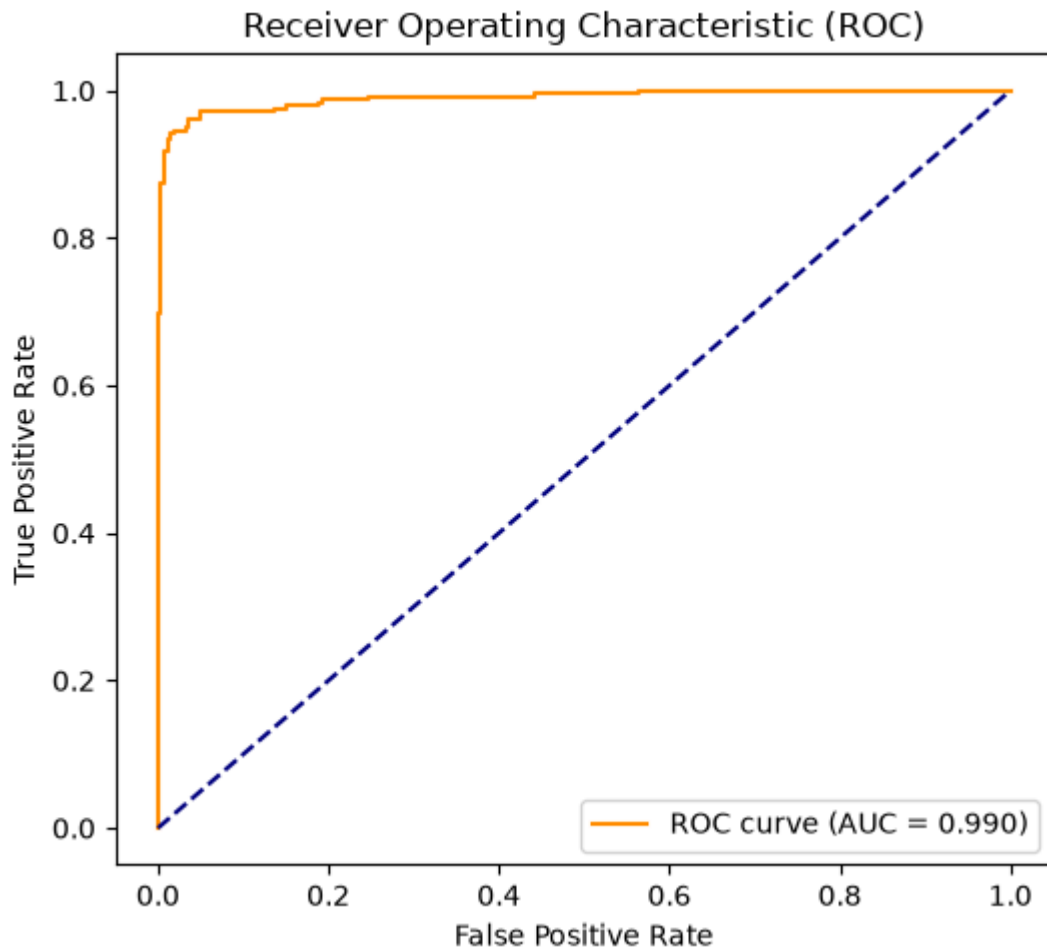
Graph 2 - Confusion Matrix

```
In [10]: import itertools
fig, ax = plt.subplots(figsize=(5,5))
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks([0,1]); ax.set_xticklabels(['Non-neoplasia', 'Neoplasia'])
ax.set_yticks([0,1]); ax.set_yticklabels(['Non-neoplasia', 'Neoplasia'])
ax.set_xlabel('Predicted label'); ax.set_ylabel('True label')
ax.set_title('Confusion Matrix - DenseNet121')
for i,j in itertools.product(range(2), range(2)):
    ax.text(j, i, cm[i,j], ha='center', va='center',
            color='white' if cm[i,j] > cm.max()/2 else 'black')
plt.colorbar(im, fraction=0.046, pad=0.04)
plt.tight_layout(); plt.savefig('densenet121_confusion_matrix.png', dpi=150); pl
```



Graph 3 - ROC Curve

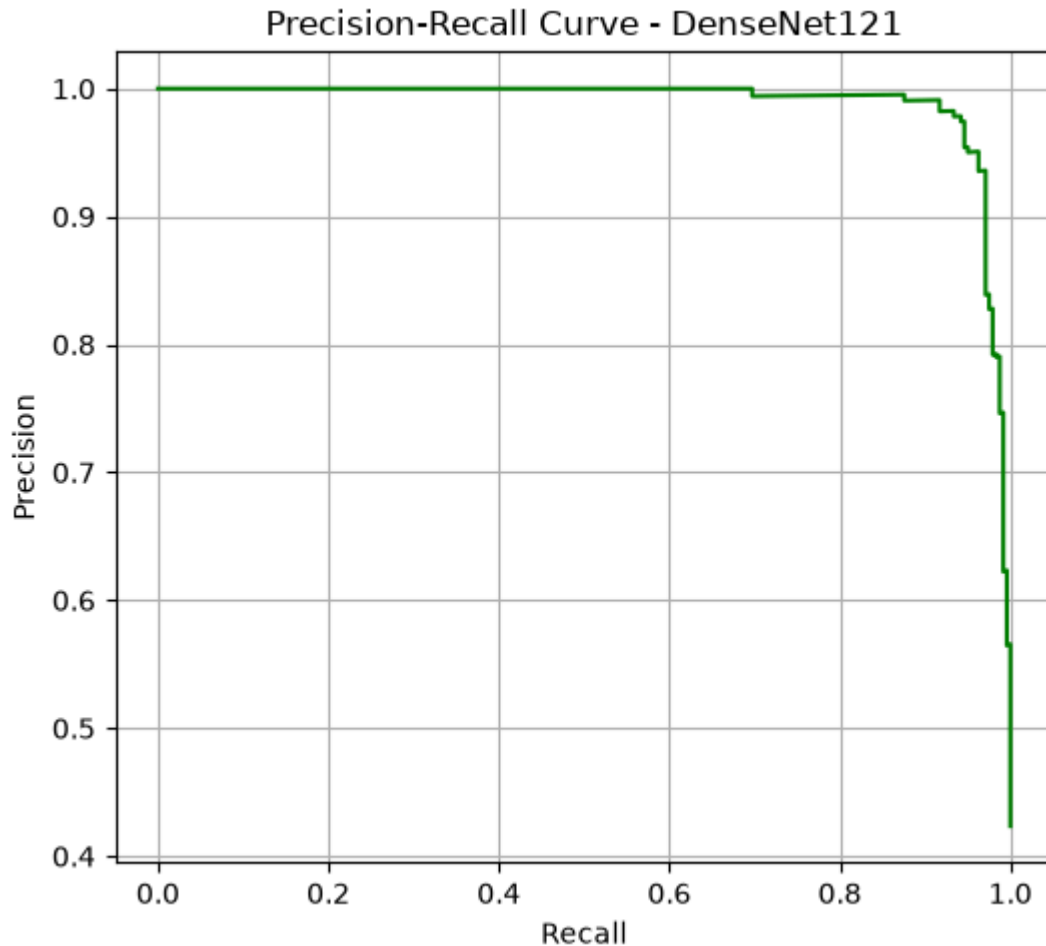
```
In [11]: plt.figure(figsize=(5.5,5))
plt.plot(fpr, tpr, color='darkorange', label=f'ROC curve (AUC = {roc_auc:.3f})')
plt.plot([0,1],[0,1], linestyle='--', color='navy')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc='lower right')
plt.tight_layout(); plt.savefig('densenet121_roc.png', dpi=150); plt.show()
```



Extra graphs for the report (not required by the exercise, but standard in medical-imaging papers)

1. **Precision-Recall curve** - more informative than ROC when the two classes are imbalanced (check your Neoplasia/Non-neoplasia counts above; CCE polyp sets usually are imbalanced).
2. **Grad-CAM** on a few test images - shows *where* the network looked; strengthens a clinical-AI report far more than accuracy numbers alone, and costs one extra cell with `torchvision` /hooks.
3. **Group comparison bar chart** - once member2/member3 numbers exist, one bar chart with all 3 models side by side (Accuracy/Sensitivity/Specificity/F1/AUC) is the figure your report actually needs for Table 1.

```
In [12]: precision, recall, _ = precision_recall_curve(test_targets, test_probs)
plt.figure(figsize=(5.5,5))
plt.plot(recall, precision, color='green')
plt.xlabel('Recall'); plt.ylabel('Precision'); plt.title('Precision-Recall Curve')
plt.grid(True); plt.tight_layout(); plt.savefig('densenet121_pr_curve.png', dpi=
```



```
In [13]: # Grad-CAM on Last conv block of DenseNet121 (features.denseblock4)
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.gradients = None
        self.activations = None
        target_layer.register_forward_hook(self._save_activation)
        target_layer.register_full_backward_hook(self._save_gradient)
    def _save_activation(self, module, inp, out):
        self.activations = out.detach()
    def _save_gradient(self, module, grad_in, grad_out):
        self.gradients = grad_out[0].detach()
    def __call__(self, x):
        self.model.zero_grad()
        output = self.model(x)
        output.backward(torch.ones_like(output))
        weights = self.gradients.mean(dim=(2,3), keepdim=True)
        cam = torch.relu((weights * self.activations).sum(dim=1, keepdim=True))
        cam = torch.nn.functional.interpolate(cam, size=x.shape[2:], mode='bilinear')
        cam = cam.squeeze().cpu().numpy()
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam

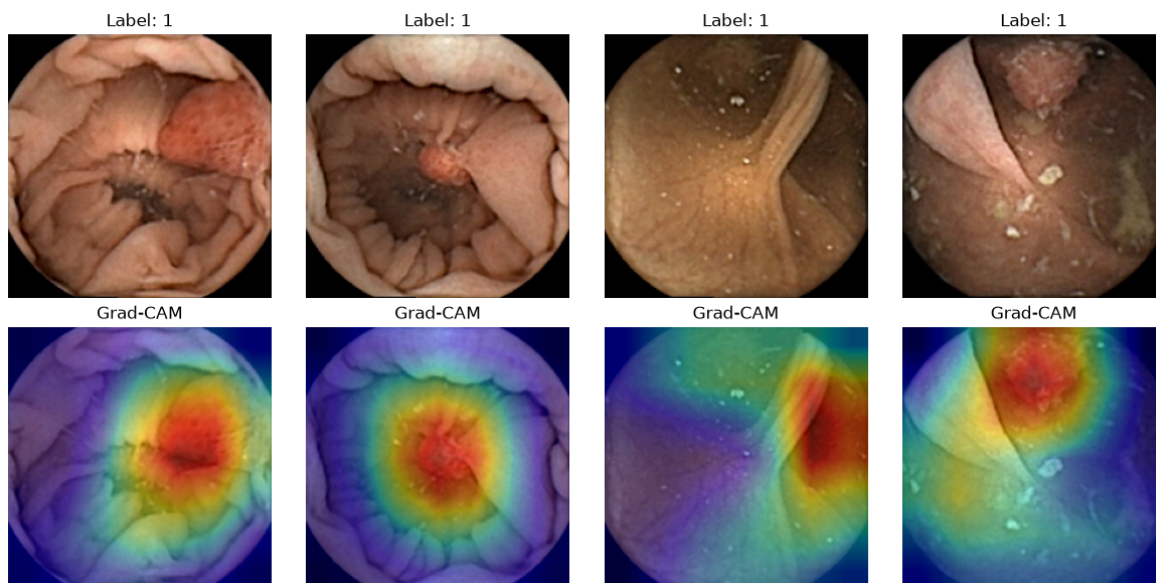
gradcam = GradCAM(pc_model, pc_model.features.denseblock4)

images, labels, names = next(iter(test_loader))
n_show = 4
fig, axes = plt.subplots(2, n_show, figsize=(3*n_show, 6))
for i in range(n_show):
    img = images[i:i+1].to(device)
```

```

cam = gradcam(img)
disp = images[i].permute(1,2,0).numpy() * 0.5 + 0.5
axes[0,i].imshow(disp); axes[0,i].set_title(f'Label: {labels[i].item()}'); a
axes[1,i].imshow(disp); axes[1,i].imshow(cam, cmap='jet', alpha=0.45); axes[
axes[1,i].set_title('Grad-CAM')
plt.tight_layout(); plt.savefig('densenet121_gradcam.png', dpi=150); plt.show()

```



Group Table 1 (fill in member2/member3 once available)

Friend's numbers below are transcribed from his results; replace **None** with the third teammate's numbers.

```

In [14]: import pandas as pd
table1 = pd.DataFrame([
    {'Model': 'ResNet18 (teammate)', 'Pretrained or Scratch': 'Scratch', 'Accur
    {'Model': 'DenseNet121 (you)', 'Pretrained or Scratch': 'Pretrained', 'Accur
    {'Model': 'TBD (member3)', 'Pretrained or Scratch': 'Pretrained', 'Accu
])
table1

```

Out[14]:

	Model	Pretrained or Scratch	Accuracy (%)	Sensitivity (%)	Specificity (%)	F1 Score	AUC
0	ResNet18 (teammate)	Scratch	70.83	93.75	25.00	0.8108	0.7205
1	DenseNet121 (you)	Pretrained	96.49	94.19	98.17	0.9578	0.9898
2	TBD (member3)	Pretrained	NaN	NaN	NaN	NaN	NaN

In []: